

Architecture Overview

As our chosen architecture, We chose MVC because it separates the user interface (View) from the business logic (Model). This allows our team (Julian, Vinay, Branson, Leo) to work on the front end and back end in parallel without conflicts, accelerating development. This is also known as the Model, View, Controller architecture, and is how our system will interact with itself.

Model (data & business logic) - Responsible for managing the data, state, and DB interaction. It also enforces rules of the application

View (User Interface) - Responsible for displaying information from the Model, to the user and sending user actions to the controller

Controller (Intermediary) - Acts as the intermediate middle man for the view and model, receiving user input from the view, and sending to the model for interactions with the DB, and then selects the view for rendering.

With our specific system, an example would include the login feature, where users are displayed a login button (from the view), clicking/interacting with the button (controller), the controller send the login information to the model where the data is validated (model), and then sent back to the controller, where the correct view is displayed based on the state of the controller (success login/fail login).

With a non functional req, (e.g/supporting 500 concurrent users) MVC supports NFR-2 (Scalability) by decoupling the View from the Model. This allows us to optimize the database queries (Model) independently of the user interface, ensuring that heavy back end processing does not freeze the user's browser experience.